In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import learning_curve

# Generate some synthetic data
np.random.seed(0)
X = np.linspace(-5, 5, 100)
y = np.sin(X) + np.random.normal(0, 0.5, 100)
X = X[:, np.newaxis]

# Plot the data to see the underlying sine wave pattern
plt.figure(figsize=(10, 6))
plt.scatter(X, y, label='Data Points')
plt.title('Synthetic Sine Wave Data with Noise')
plt.legend()
plt.show()

# --- ENTER YOUR CODE HERE ---
plt.figure(figsize=(14, 8))
plt.scatter(X, y, label='Data Points')
degrees = [1, 4, 15]
for degree in degrees:
    # 1. Create a pipeline that adds polynomial features and then fits a
linear regression model
    model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
    # 2. Train the model
    model.fit(X, y)
    # 3. Make predictions
    y_pred = model.predict(X)
    # 4. Plot the model's predictions
    plt.plot(X, y_pred, label=f'Degree {degree} (Model)')
plt.legend()
plt.title('Comparing Model Complexities')
plt.ylim(-3, 3)
plt.show()

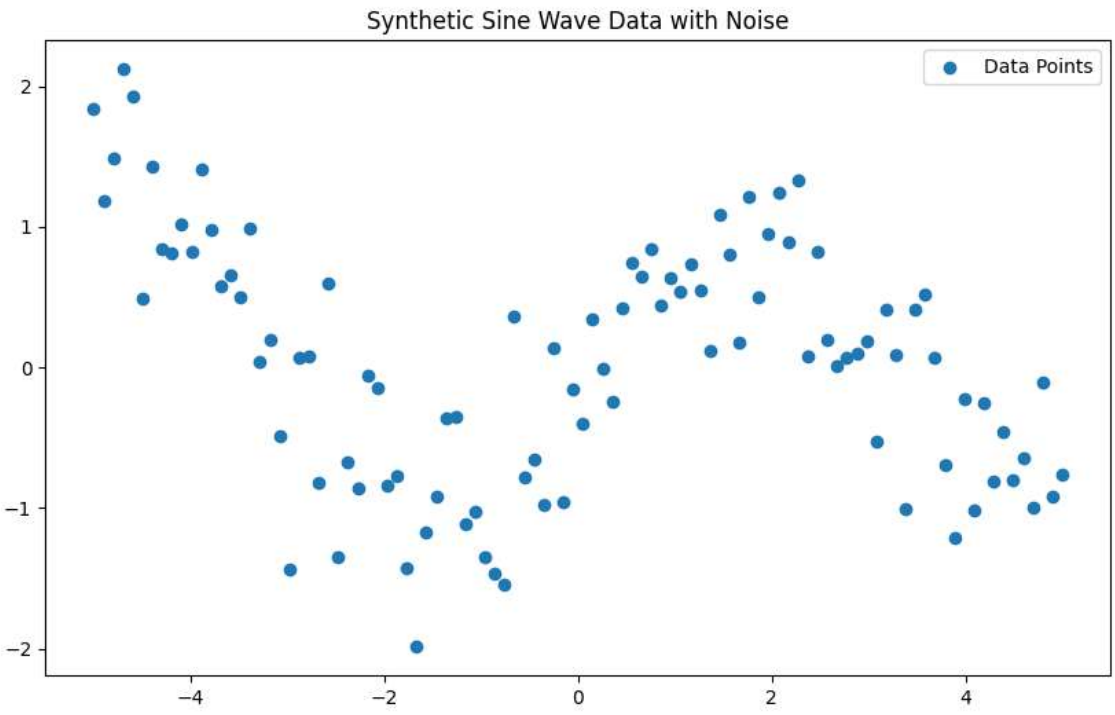
def plot_learning_curve(estimator, title, X, y, ylim=None, cv=None,
n_jobs=None, train_sizes=np.linspace(.1, 1.0, 5)):
    plt.figure(figsize=(10,6))
    plt.title(title)
    if ylim is not None:
        plt.ylim(*ylim)
    plt.xlabel("Training examples")
    plt.ylabel("Score")
    train_sizes, train_scores, test_scores = learning_curve(
        estimator, X, y, cv=cv, n_jobs=n_jobs, train_sizes=train_sizes)
    train_scores_mean = np.mean(train_scores, axis=1)
    train_scores_std = np.std(train_scores, axis=1)
    test_scores_mean = np.mean(test_scores, axis=1)
```

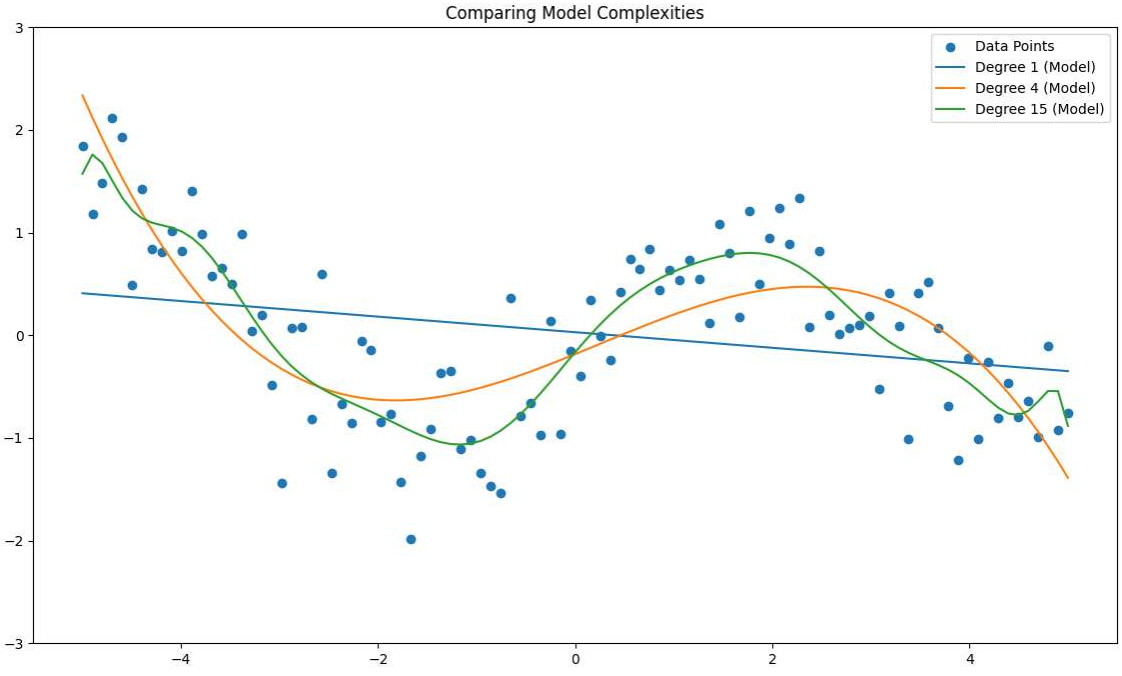
```
    test_scores_std = np.std(test_scores, axis=1)
    plt.grid()
    plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                     train_scores_mean + train_scores_std, alpha=0.1,
                     color="r")
    plt.fill_between(train_sizes, test_scores_mean - test_scores_std,
                     test_scores_mean + test_scores_std, alpha=0.1,
color="g")
    plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
             label="Training score")
    plt.plot(train_sizes, test_scores_mean, 'o-', color="g",
             label="Cross-validation score")
    plt.legend(loc="best")
    return plt

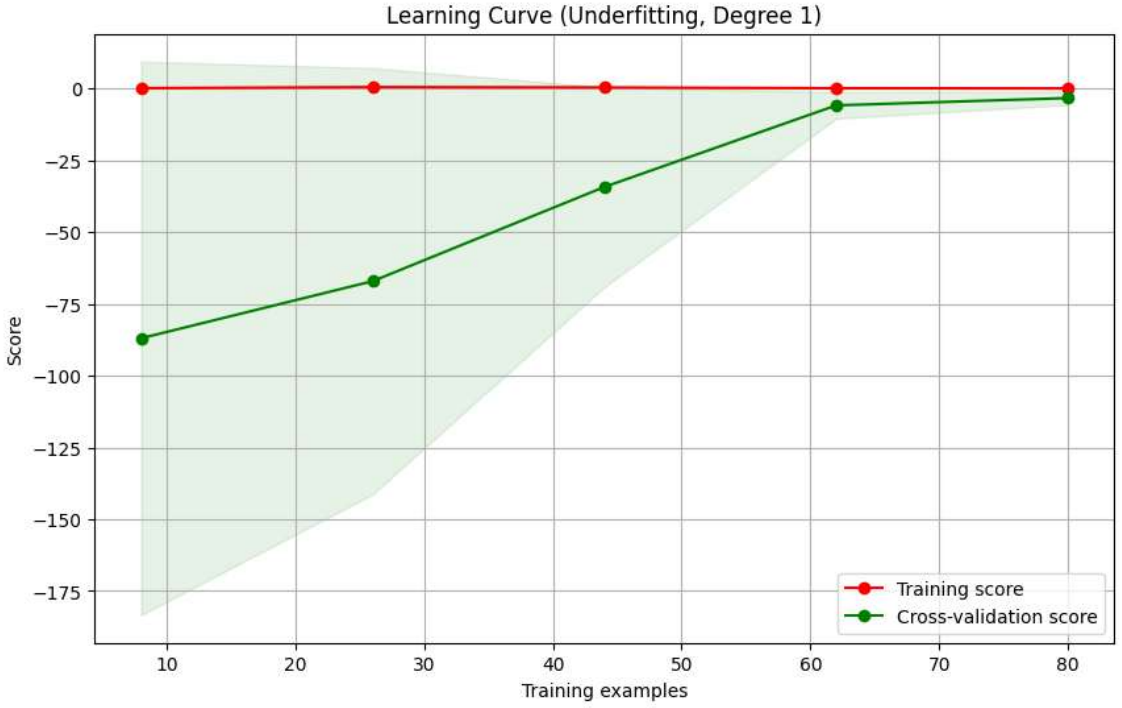
# --- ENTER YOUR CODE HERE ---
# 1. Create the underfit and overfit models
underfit_model = make_pipeline(PolynomialFeatures(1), LinearRegression())
overfit_model = make_pipeline(PolynomialFeatures(15), LinearRegression())

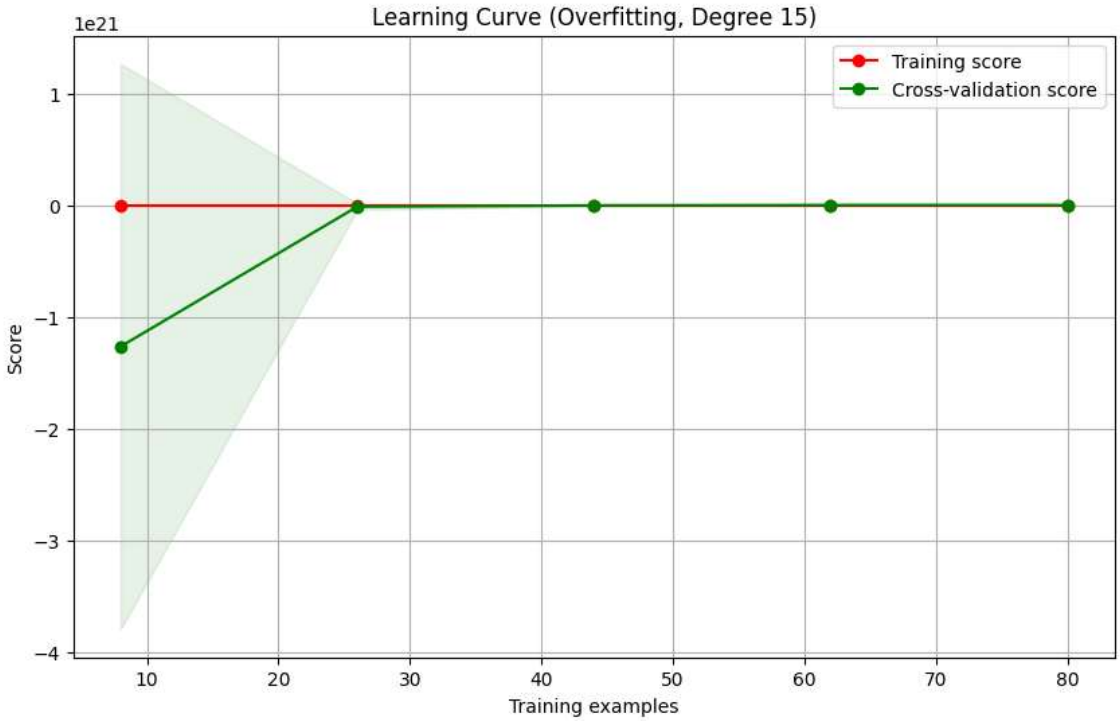
# 2. Plot the learning curve for the underfit model
plot_learning_curve(underfit_model, "Learning Curve (Underfitting, Degree
1)", X, y, cv=5)
plt.show()

# 3. Plot the learning curve for the overfit model
plot_learning_curve(overfit_model, "Learning Curve (Overfitting, Degree
15)", X, y, cv=5)
plt.show()
```




Comparing Model Complexities
Data Points
Degree 1 (Model)
Degree 4 (Model)
Degree 15 (Model)
3
2
1
0
−1
−2
−3
−4
−2
0
2
4


Learning Curve (Underfitting, Degree 1)
Score
0
−25
−50
−75
−100
−125
−150
−175
10
20
30
40
50
60
70
80
Training examples
Training score
Cross-validation score



Knowledge Check

1. In the first plot of the three models, which model (degree 1, 4, or 15) is underfitting, which is overfitting, and which is a good fit? Explain your reasoning

The Degree 1 model underfits the data due to its high simplicity (a straight line), resulting in high bias and poor performance. The Degree 4 model strikes an ideal balance, capturing the true underlying sine wave pattern accurately. In contrast, the Degree 15 model overfits by being excessively complex, memorizing random noise and individual data points rather than the general trend, which leads to high variance.

2. Looking at the learning curve for the underfitting model, what do you observe about the training and cross-validation scores? What does this tell you?

The learning curve for the underfitting model displays both low training and cross-validation scores that plateau early, even as more data is introduced. This behavior signifies high bias, indicating that the model is fundamentally too simple to learn the underlying pattern and that providing additional training examples will not improve its predictive power.

3. Looking at the learning curve for the overfitting model, what do you observe about the gap between the training and cross-validation scores? What does this tell you?

The learning curve for the overfitting model is characterized by a large gap between a very high training score and a significantly lower cross-validation score. This divergence indicates high variance, showing that while the model memorizes training data and noise exceptionally well, it fails to generalize effectively to unseen validation data.